

Lecture 2

Introduction to Computers and Programming in C

DataTypes



Introduction to C

- C is a procedural programming language.
- It was initially developed by Dennis Ritchie in the year 1972.
- It was mainly developed as a system programming language to write an operating system.



Introduction to C

- The Unix operating system and virtually all Unix applications are written in the C language. C has now become a widely used professional language for various reasons.
- Easy to learn
- Structured language.
- It can handle low-level activities.
- It can be compiled on a variety of computers.



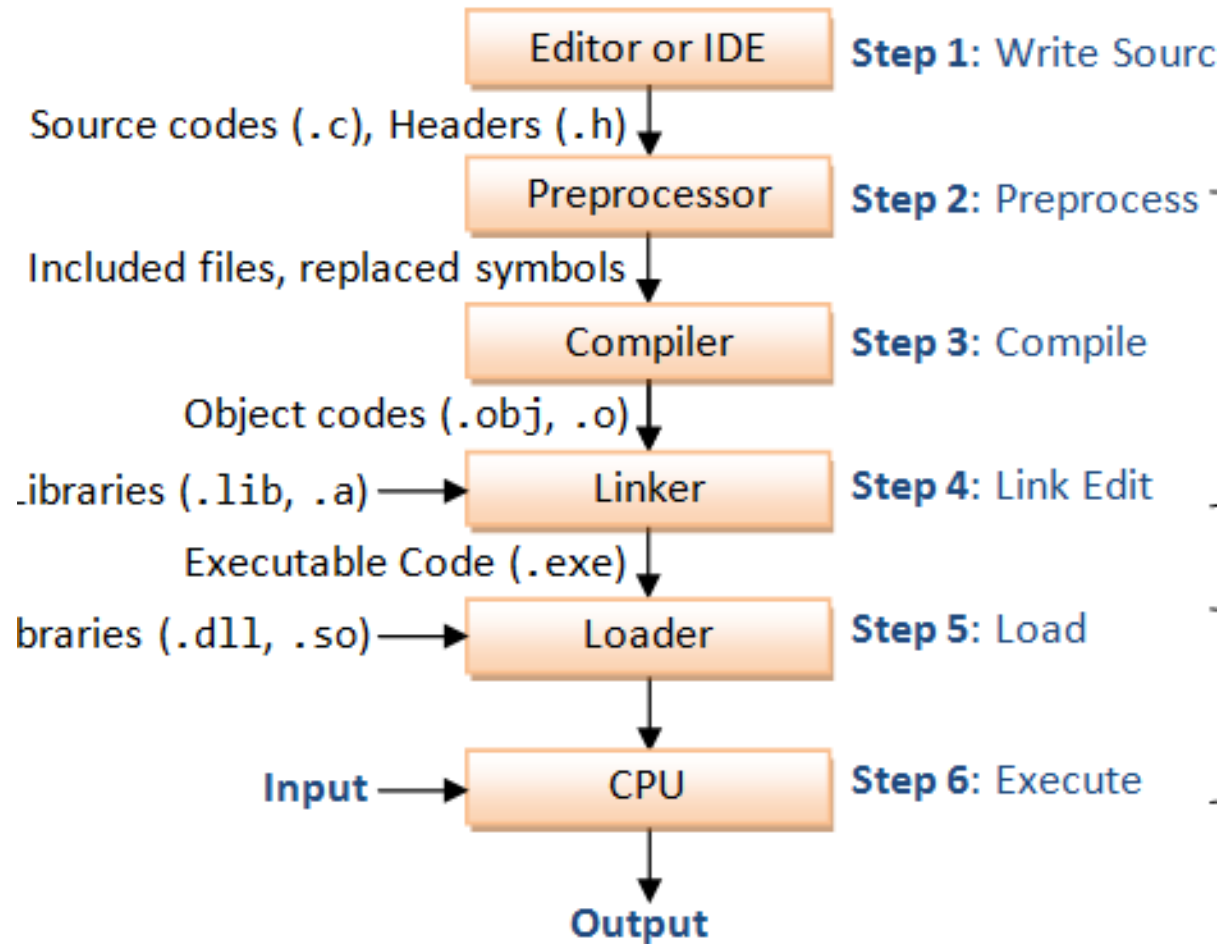
Facts about C

- C is a successor of B language which was introduced around 1970
- The language was formalized in 1988 by the American National Standard Institute (ANSI).
- By 1973 UNIX OS almost totally written in C.
- Today C is the most widely used System Programming Language.

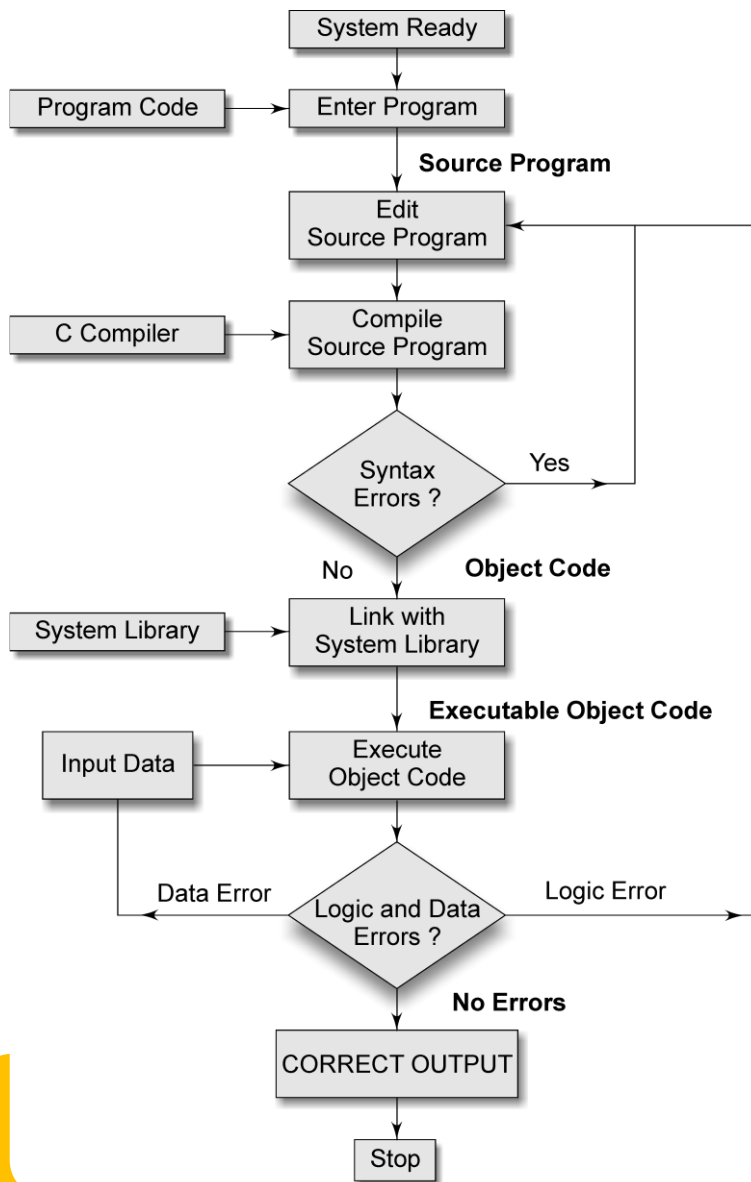


Why to use C

- C was initially used for system development work, in particular the programs that make-up the operating system. Some examples of the use of C might be:
- Operating Systems
- Language Compilers
- Assemblers
- Text Editors
- Print Spoolers
- Network Drivers
- Data Bases
- Language Interpreters



C- Program Structure



Process of Compiling a 'C' Program

Important Points

- C is a case sensitive programming language. In C *printf* and *Printf* will have different meanings.
- End of each C statement must be marked with a semicolon.
- Multiple statements can be one the same line.

Important Points

- White Spaces (ie tab space and space bar) are ignored.
- Statements can continue over multiple lines
- **Comments:** are used to give additional useful information inside a C Program. All the comments will be put inside `/*...*/` (*Multi-line Comment*) or `//` (*End-of-line Comment*)

Preprocessor directive

`#include <stdio.h>`

The "#include" is called a preprocessor directive. A preprocessor directive begins with a # sign, and is processed before compilation. The directive "#include <stdio.h>" tells the preprocessor to include the "stdio.h" header file to support input/output operations.

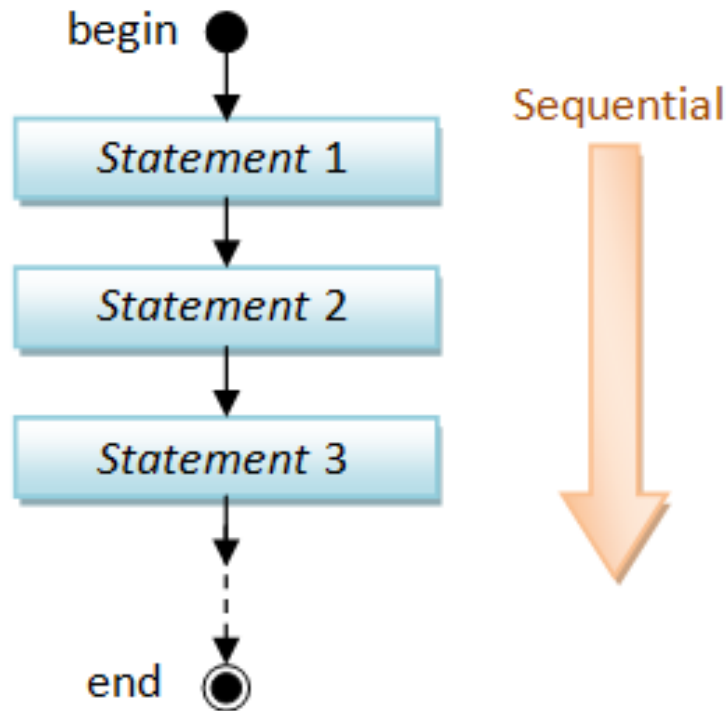
```
int main() { ..... }
```

- The main() function is the entry point of program execution. main() is required to return an int (integer).

C Program Template

```
/* Comment to state the purpose of this program (filename.c) */  
#include <stdio.h>  
int main()  
{  
    // Your Programming statements HERE!  
    return 0;  
}
```

Program



- A *program* is a *sequence of instructions* (called *programming statements*), executing one after another usually in a *sequential* manner

Online C Compiler

https://www.onlinegdb.com/online_c_compiler

First Sample Program

```
/* First C program (Hello.c) */
#include <stdio.h> // Needed to perform IO operations
int main() {      // Program entry point
printf("Hello, world!\n"); // Says Hello
return 0;         // Terminate main()
}                // End of main()
```

```
printf("Hello, world!\n");
```

- Invoke the function `printf()` to print the string "Hello, world!" followed by a newline (`\n`) to the console.
- The newline (`\n`) brings the cursor to the beginning of the next line.

Return statement

- `return 0;`

terminates the `main()` function and returns a value of 0 to the operating system. Typically, return value of 0 signals normal termination; whereas value of non-zero (usually 1) signals abnormal termination. This line is optional. C compiler will implicitly insert a `"return 0;"` to the end of the `main()` function.

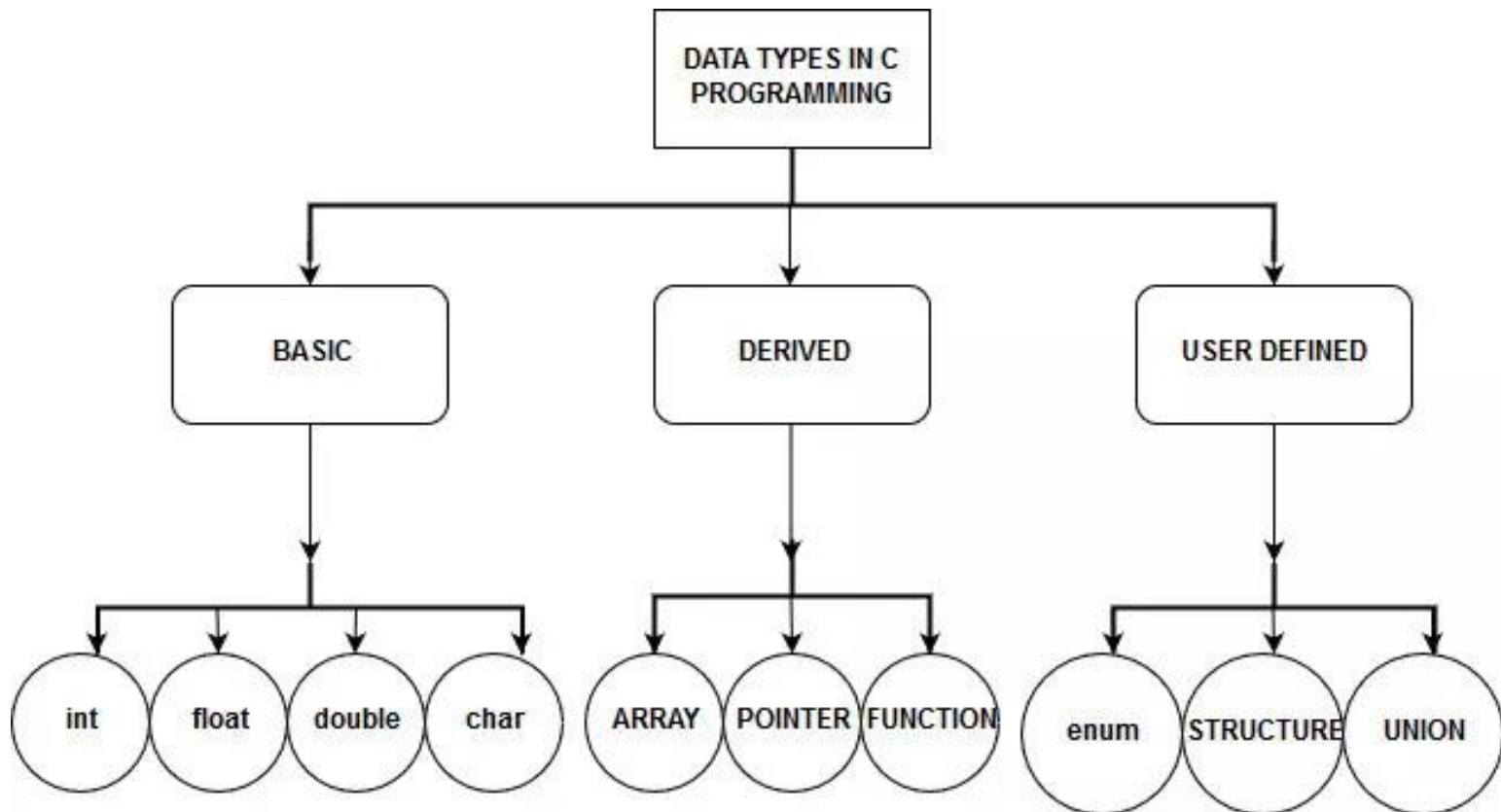
Case Sensitive

- C is a case sensitive Language.

Variable

- Computerprograms *manipulate* (or *process*) data.
- A *variable* is used to store a piece of data for processing. It is called *variable*.
- A *variable* is a *named* storage location, that stores a *value* of a particular data *type*. In other words, a *variable* has a *name*, a *type* and stores a *value* of that type.

Data Types



Basic Data Types

Int (Integer)

Store integer type of data.

- **2 bytes (16 bits)** on a **16-bit machine**.
- The modern systems have **32 or 64-bit configurations**. The size of an integer in such environment is **4 bytes** with a **starting value of -2,147,483,648** and an **ending value as 2,147,483,647**.

Basic Data Types

Float

- This datatype is also numeric but allows numbers with decimal places. It has a size of **4 bytes** and has a range of **3.4E-38 to 3.4E+38**.
- The float type has a precision up to 7 digits.

Basic Data Type

Double

- It is same as float but with a higher precision range that is **8 bytes** which gives it an ability to store more numerals after the decimal places, double to be exact.
- It has a length of **8 bytes** and has a range of **$1.7E \pm 308$** . The double data type has a precision up to 15 digits.

Basic Data Types

Char

Char data type by default allows a single character such as a letter from the alphabet. It has the size of just **1 byte** with a range of **-128 to 127** or **0 to 255**.

TYPE	SIZE (Bits)	Range
char or signed char	8	-128(-2^7) to 127(2^7-1)
unsigned char	8	0 to 255
int or signed int	16	-32768(-2^{15}) to 32767 ($2^{15} - 1$)
unsigned int	16	0 to 65535
short int or signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or signed long int or long	32	-2147483648 to 2147483647
unsigned long int	32	0 to 4294967295
float	32	3.4 e-38 to 3.4 e+38
double	64	1.7e-308 to 1.7e+308
long double	80	3.4 e-4932 to 3.4 e+4932

Size and Range of Data Types

Table 4.9 *Data Types and Their Keywords*

<i>Data type</i>	<i>Keyword equivalent</i>
Character	char
Unsigned character	unsigned char
Signed character	signed char
Signed integer	signed int (or int)
Signed short integer	signed short int (or short int or short)
Signed long integer	signed long int (or long int or long)
Unsigned integer	unsigned int (or unsigned)
Unsigned short integer	unsigned short int (or unsigned short)
Unsigned long integer	unsigned long int (or unsigned long)
Floating point	float
Double-precision floating point	double
Extended double-precision floating point	long double



```
#include <stdio.h>
```

```
int main() {
```

```
int number1, number2, sum;
```

```
printf("Enter two integers: ");
```

```
scanf("%d %d", &number1, &number2);
```

```
// calculate the sum
```

```
sum = number1 + number2;
```

```
printf("%d + %d = %d", number1, number2, sum);
```

```
return 0;
```

```
}
```

Program

WAP to find area of
circle

WAP to divide two
numbers

WAP to add two
integers

WAP to print ASCII
value

WAP to multiply
floating point numbers

WAP to find area of circle

- `#include <stdio.h>`
- `int main(void) {`
- `float pie = 3.14;`
- `int radius = 6;`
- `printf("The radius of the circle is %d " , radius);`
- `float area = (float)(pie* radius * radius);`
- `printf("The area of the given circle is %f", area);`
- `return 0;`
- `}`

- `#include <stdio.h>`
- `int main(void) {`
- `float pie = 3.14;`
- `int radius = 6;`
- `printf("The radius of the circle is %d " , radius);`
- `float area = (float)(pie* (pow(radius,2)));`
- `printf("The area of the given circle is %f", area);`
- `return 0;`
- `}`

- `#include <stdio.h>`
- `#include <conio.h> #define PI 3.141`
- `int main()`
- `{`
- `float radius, area;`
- `printf("Enter radius of circle\n");`
- `scanf("%f", & radius);`
- `area = PI * radius * radius;`
- `printf("Area of circle : %0.4f\n", area);`
- `getch();`
- `return 0;`
- `}`

WAP to divide two numbers

```
#include<stdio.h>
int main()
{
    int num1,num2,div;
    printf("\nEnter Two Numbers\n");
    printf("-----\n");
    printf("Enter First Number : ");
    scanf("%d", &num1);
    printf("\nEnter Second Number : ");
    scanf("%d",&num2);
    div=num1/num2;
    printf("\nDivision of %d & %d is = %d",num1,num2,div);
    return 0;
}
```


WAP to print ASCII value

In C programming, a character variable holds ASCII value (an integer number between 0 and 127) rather than that character itself. This integer value is the ASCII code of the character.

For example, the ASCII value of 'A' is 65.

What this means is that, if you assign 'A' to a character variable, 65 is stored in the variable rather than 'A' itself.

```
#include <stdio.h>

int main() {
    char c;
    printf("Enter a character: ");
    scanf("%c", &c);

    // %d displays the integer value of a
    character

    // %c displays the actual character
    printf("ASCII value of %c = %d", c, c);

    return 0;
}
```

WAP to multiply floating point numbers

```
#include <stdio.h>

int main() {
    double a, b, product;
    printf("Enter two numbers: ");
    scanf("%lf %lf", &a, &b);

    // Calculating product
    product = a * b;

    // %.2lf displays number up to 2 decimal point
    printf("Product = %.2lf", product);

    return 0;
}
```